

PROJECT SUMMARY REPORT

NetSage AI

AI-Assisted Network Troubleshooting with Human Review

NAME – PRINCE BANSAL

COLLEGE – MIT AOE , ALANDI , PUNE

GROUP SIZE – 1

CONTRIBUTION - 100% developed and completed by Prince Bansal

1. Introduction

NetSage AI is an AI-assisted network troubleshooting system developed to help identify common networking problems in Cisco Packet Tracer lab scenarios.

While working with networking, it can sometimes be difficult to understand the actual reason behind a problem such as a PC not being able to communicate with a server. There can be different reasons like a wrong IP address, wrong subnet mask, wrong gateway, or a router interface problem.

The main purpose of NetSage AI is to take the available network evidence, analyze it, identify the most likely fault, provide a confidence level, explain the reason, and suggest the corrective action.

The system does not automatically make changes to the network. The recommendation is reviewed by a human, after which the fix can be applied manually and verified.

2. Objective

The main objectives of this project are:

- To develop a simple AI-assisted network troubleshooting system.
- To identify common network configuration problems.
- To use network evidence for finding the probable root cause.
- To provide a confidence value for the prediction.
- To use machine learning for fault classification.

- To use SHAP to identify the major factor contributing to the prediction.
- To provide a recommended corrective action.
- To keep a human reviewer involved before applying the fix.
- To verify whether the problem is solved after the correction.
- To generate reports and maintain a record of previous reviews.

3. Problem Statement

Junior network engineers and students may know individual networking commands but can still face difficulty in connecting a network symptom with its actual root cause.

For example, when a PC cannot reach a server, the problem could be related to an incorrect IP address, subnet mask, gateway, interface status, or router configuration.

NetSage AI was developed to assist with this type of troubleshooting by analyzing the available evidence and suggesting the most likely problem and corrective action.

4. Project Scope

For this project, five practical network troubleshooting cases were implemented and tested.

The cases are:

1. CASE01 – Wrong Subnet Mask
2. CASE02 – Interface Down
3. CASE03 – Wrong Gateway
4. CASE04 – Wrong IP Address
5. CASE05 – Wrong Router Interface IP

These cases were selected to demonstrate the complete troubleshooting workflow without making the project unnecessarily large.

5. Working of the System

The overall working of NetSage AI is:

Network Problem → Evidence Collection → Rule-Based Check → ML Prediction → SHAP Explanation → Diagnosis → Human Review → Manual Fix → Verification → Report

First, the network problem and relevant evidence are provided to the system.

The rule-based Python checker checks common configuration problems. The Random Forest model is then used to predict the probable fault.

After the prediction, SHAP is used to understand which input feature contributed the most to the prediction.

The system then displays the fault, confidence, reason, and recommended action.

A human reviewer checks the recommendation before the actual fix is applied. After applying the fix manually, the network is tested again to verify whether the problem has been solved.

6. Technologies Used

The main technologies and tools used in the project are:

- **Python** – Main programming language
- **Scikit-learn** – Machine learning and Random Forest
- **SHAP** – Model explanation
- **JSON** – Case and review data storage
- **Packet Tracer** – Network troubleshooting scenarios
- **Pandas** – Dataset handling
- **Joblib** – Saving the trained ML model
- **Python file handling** – Reports and logs

7. Rule-Based Troubleshooting

A Python-based rule checker was created to identify common network configuration problems.

The system checks evidence such as:

- Packet loss
- Router interface status
- Gateway match
- IP address match
- Subnet mask match

Based on these inputs, the system can identify faults such as:

- Wrong Subnet Mask

- Interface Down
- Gateway Mismatch
- Wrong IP Address
- Wrong Router Interface IP

The rule-based approach makes the basic troubleshooting process deterministic and easy to understand.

8. Machine Learning Component

A Random Forest classification model was added to make the system more AI-based.

A dataset containing network troubleshooting examples was created and used to train the model.

The model was trained to classify the five fault categories.

The final trained model achieved 100% accuracy on the prepared test dataset.

The trained model was saved as:

netsage_model.pkl

The model prediction is displayed along with its confidence.

9. SHAP Explainability

SHAP was added to make the machine learning prediction easier to understand.

Instead of only showing the predicted fault, the system also shows the contribution of different input features.

For example, in one test the system predicted:

Wrong Router Interface IP

The major contributing factor was:

router_ip_match

This helps explain why the model made a particular prediction instead of treating the ML model as a complete black box.

10. Human Review

Human review is an important part of NetSage AI.

The system does not automatically apply the recommended network correction.

After the diagnosis, the reviewer is asked:

“Do you approve this recommended action?”

The reviewer can approve or reject the recommendation.

If approved, the recommended fix is applied manually and the network is tested again.

The verification result is then stored in the review log.

This keeps the final network decision under human control.

11. Report Generation

The system automatically generates a report after a diagnosis.

The report contains:

- Date and time
- Network evidence
- Detected fault
- Confidence
- Reason
- Recommended action
- Verification instructions

Reports are stored inside the reports folder with the corresponding case ID and timestamp.

This makes it easier to keep a record of troubleshooting activities.

12. Dashboard

A dashboard was also developed to summarize the troubleshooting results.

The dashboard displays information such as:

- Total reviewed cases
- Different issue types
- Human approvals
- Human rejections
- Successful verification

- Failed verification
- Reports generated

This gives a quick overall view of the troubleshooting activity.

13. Testing

All five implemented cases were tested individually.

The test cases were:

Case	Fault
CASE01	Wrong Subnet Mask
CASE02	Interface Down
CASE03	Wrong Gateway
CASE04	Wrong IP Address
CASE05	Wrong Router Interface IP

The cases were tested through the troubleshooting system, and the diagnosis, recommendation, human review, and verification workflow were checked.

14. Result

The completed NetSage AI system is able to:

- Analyze network troubleshooting evidence.
- Identify common configuration faults.
- Predict the probable fault using Random Forest.
- Provide prediction confidence.
- Explain the important contributing factor using SHAP.
- Suggest a corrective action.
- Allow human approval or rejection.
- Record verification results.
- Generate troubleshooting reports.
- Display summarized results through a dashboard.

The system successfully demonstrates the complete workflow from identifying a broken network scenario to diagnosing, reviewing, fixing, and verifying the problem.

15. Conclusion

NetSage AI was developed as an individual project to demonstrate how AI and traditional rule-based troubleshooting can be used together for network troubleshooting.

The project combines Packet Tracer scenarios, Python rules, machine learning, SHAP explainability, human review, verification, reporting, and dashboard functionality.

The main focus of the project is not to automatically change the network, but to assist the user in finding the problem and suggesting the next action while keeping the final decision with the human reviewer.

The five implemented cases provide a practical demonstration of the complete system and show how the same approach can be extended to more networking problems in the future.